

Literature-Comparable Itinerary Recommendation

Strategy D — the Flickr photo-trajectory benchmark

*From non-comparable Foursquare-NYC scores to pairs-F1 on the same scale as
Chen 2016, DeepTrip, SelfTrip and AR-Trip*

Detailed technical document · companion to the slide deck

Repository: github.com/6ym6n/PFE_IMPLEMENTATION

Executive summary

Phase 2 of this thesis built an **itinerary recommender** — given a start POI, an end POI and a desired length K , produce an **ordered route** of POIs — and evaluated it on Foursquare NYC, scoring **pairs-F1 $\approx 0.26\text{--}0.29$** . The published trip-recommendation literature reports pairs-F1 $\approx 0.6\text{--}0.8$, which made our numbers look weak. **They are not weak — they were simply not comparable.** The literature is evaluated on a different benchmark (the small **Flickr** photo-trajectory city datasets) under a specific protocol.

Strategy D re-runs the itinerary task on those exact datasets under the exact published protocol, so the numbers land on the **same scale** as the papers. Using the canonical trajectory files (Chen 2016's own preprocessing output) our trajectories are *identical* to the literature's, and a **Random baseline reproduces Chen 2016 within noise** — proving the pipeline is faithful. Our classical methods reach **pairs-F1 0.23–0.59** and the learned GCN+pointer model 0.31–0.49: both squarely on the published 0.26–0.85 scale. This document explains every piece — data, task, metric, protocol, methods, results and honest limitations.

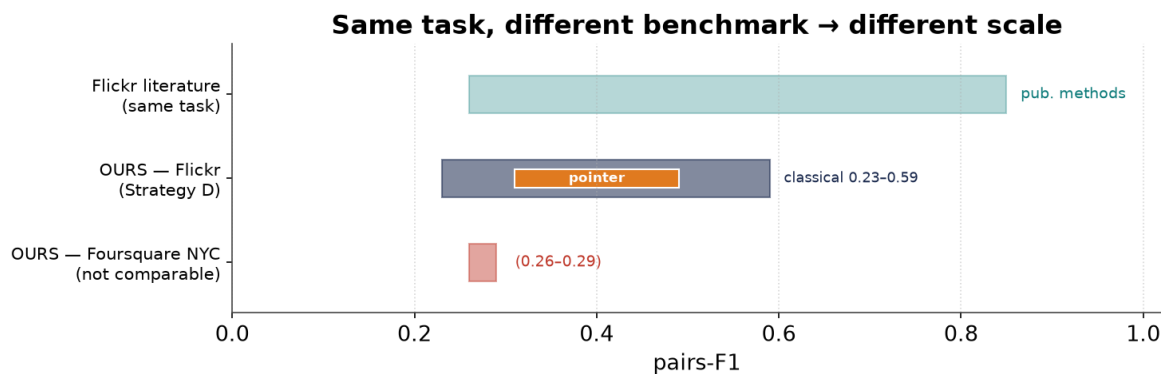


Figure 1 — The whole point in one picture: the same itinerary task on a different benchmark sits on a different scale. Strategy D puts us on the literature's scale.

1. Where Strategy D fits

The thesis addresses **two formally distinct sub-tasks**, each with its own field-standard benchmark and metrics (this separation is exactly what the tour-recommendation surveys prescribe): **next-POI prediction** on Foursquare/Gowalla check-ins (ranking metrics), and **itinerary / tour recommendation** on Flickr photo-trajectories (F1 and pairs-F1). Using a different dataset per task is correct and standard; the only error would be to cross-compare them.

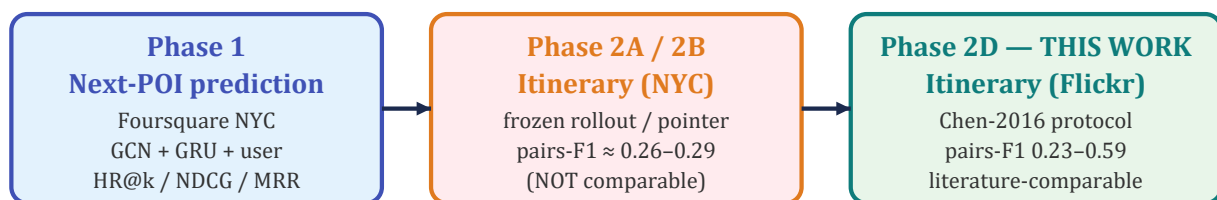


Figure 2 — Phase 1 (next-POI) feeds Phase 2 (itinerary). Phase 2A/2B ran on Foursquare NYC and produced an internal A-vs-B study; Phase 2D (this work) runs on Flickr for literature comparability.

2. The comparability problem

Two pairs-F1 numbers may be compared **only if three things match**: the **dataset** (same trajectories, same POI vocabulary size — a 30-POI city and a 5,000-POI city have completely different chance levels), the **protocol** (same split, same query, same length filter), and the **metric** (the exact same pairs-F1 definition). Foursquare NYC differs from the Flickr benchmark on all three, so its 0.26–0.29 is incomparable to the literature's 0.6–0.8 — the gap is a data/protocol artefact, not a model-quality gap (cf. Krichene & Rendle, KDD 2020, on why cross-protocol metrics are inconsistent).

- **Dataset**: Flickr cities have 27–88 POIs; NYC has ~5,000. Recovering the order of 5 of 30 POIs is a very different problem from 5 of 5,000.
- **Protocol**: the literature uses leave-one-trajectory-out CV with the origin and destination given; our NYC study used a fixed test split.
- **Metric scale**: both report pairs-F1, but on different vocabularies the achievable range differs.

3. The Flickr datasets

The benchmark is five city datasets mined from geo-tagged Flickr photos (Lim et al.; the standard data for classical and deep tour recommendation). We use the canonical `traj-{City}.csv` + `poi-{City}.csv` files — **Chen 2016's own preprocessing output**, mirrored in the `tour-cikm16` and `DeepTrip` repos — so our trajectories are *identical* to the published ones and there is zero preprocessing ambiguity. A **trajectory** is the sequence of POIs of one photo-trip (`trajID` / `seqID`), ordered by time, with consecutive duplicate POIs merged. Every trajectory in this data is **loop-free** (distinct POIs), which the pairs-F1 metric assumes.

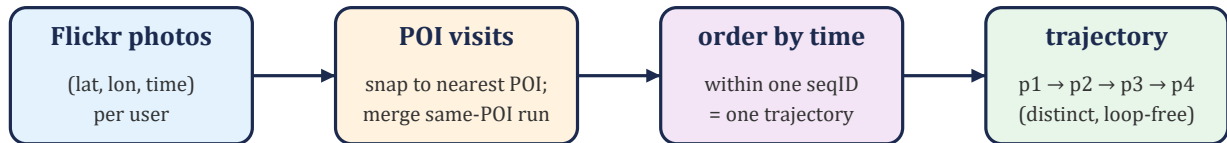


Figure 3 — From raw geo-tagged photos to an ordered, loop-free POI trajectory. The evaluated set keeps trajectories of length ≥ 3 .

City	#POIs	#users	#traj	#traj ≥ 3 (eval)
Toronto	29	1,395	6,057	335
Osaka	27	450	1,115	47
Glasgow	27	601	2,227	112
Edinburgh	28	1,454	5,028	634
Melbourne	88	1,000	5,106	442

Table 1 — Dataset statistics. The right column (length ≥ 3) is the evaluated set; Chen 2016's total-trajectory counts match the #traj column exactly, confirming we have the same data.

4. The task, precisely

A **query** is $q = (\text{start POI } s, \text{end POI } e, \text{length } K)$. The recommender must output an ordered, loop-free route of exactly K POIs that starts at s and ends at e , recovering the intermediate POIs (and their order) of the user's real trajectory. The ground truth is the held-out trajectory itself. This is the standard PersTour / Chen 2016 convention: *recover the visited sequence given its endpoints and length*.

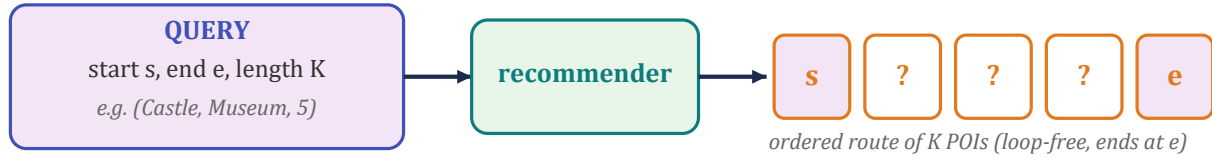
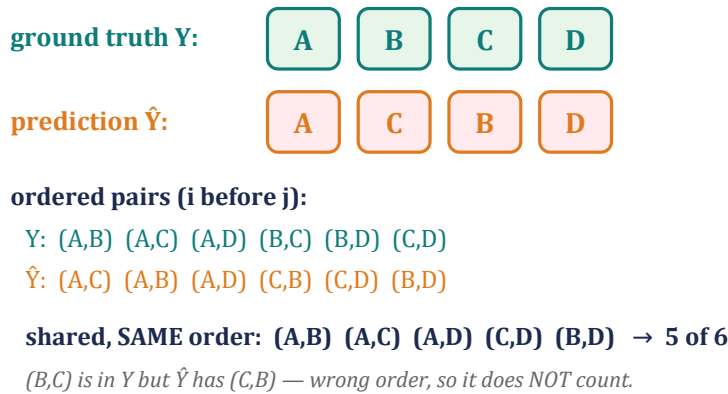


Figure 4 — The query gives the endpoints and the length; the model fills the ordered middle. Start and end are given, so a perfect recovery scores 1.0 and only the middle ordering is at stake.

5. The metric: F1 and pairs-F1

Two metrics, both in $[0, 1]$. **Point-F1** (set-F1) measures *did we pick the right POIs*, ignoring order: the F1 of the predicted POI set against the true set. **pairs-F1** (Chen 2016, the primary metric) measures *order*: it is the F1 over **ordered pairs** of POIs. For every pair (a, b) , it counts as correct only if a appears before b in *both* the prediction and the ground truth. The harmonic mean of pair-precision and pair-recall is pairs-F1.



$$\text{precision} = 5/6, \text{ recall} = 5/6 \rightarrow \text{pairs-F1} = 2 \cdot P \cdot R / (P + R) = 5/6 \approx 0.833$$

Figure 5 — A worked pairs-F1. Swapping the two middle POIs breaks exactly one ordered pair (B,C), so 5 of 6 pairs survive → pairs-F1 = $5/6 \approx 0.833$. We reuse the thesis's unit-tested implementation, pinned against Chen 2016's reference `calc_pairsF1`.

6. The protocol: leave-one-trajectory-out CV

These datasets are tiny (47–634 evaluated trajectories per city), so the field uses **leave-one-trajectory-out cross-validation**: hold out one trajectory for testing, train on all the others, predict the held-out one from its (s, e, K) query, score it, and repeat for every trajectory — then average.

We train each fold on the other **length ≥ 3** trajectories (matching the papers' folds); the held-out trajectory never influences training, its graph, or its statistics (no leakage).

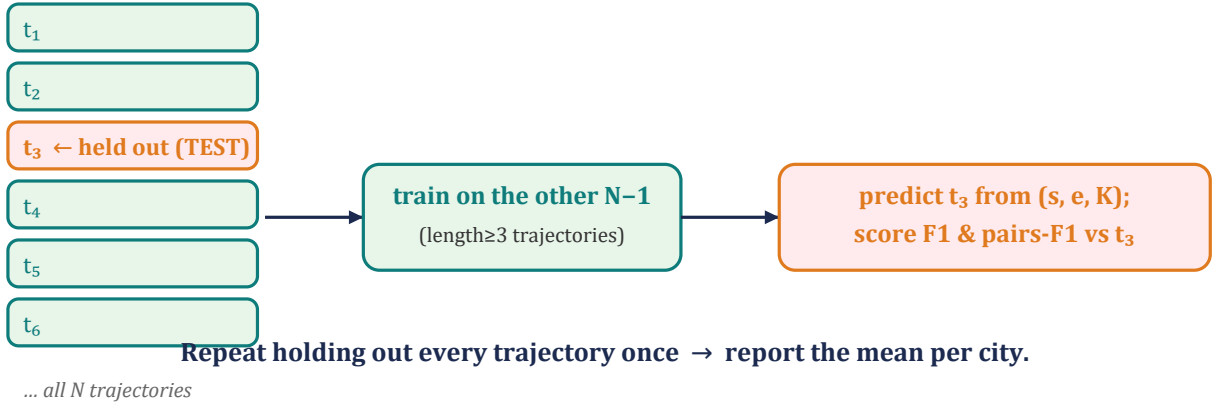
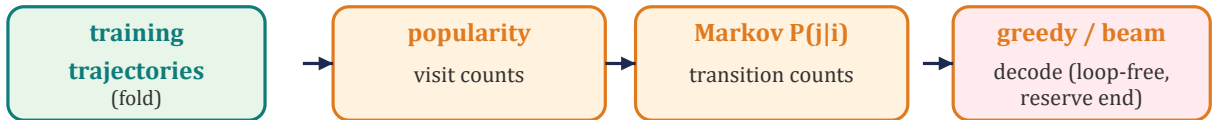


Figure 6 — Leave-one-out CV. Each trajectory is the test case exactly once; the reported number is the mean over all folds.

7. The methods we built

Two families. **Classical baselines** (fast, CPU, faithful re-implementations of Chen 2016's): **Random** (random order of the middle POIs — the floor), **PoiPopularity** (pick the most-visited POIs, ordered by popularity), **Markov** (a first-order POI→POI transition model decoded greedily) and **MarkovPath** (the same model decoded by beam search for the most-likely loop-free path). The **learned model** is a light, self-contained **GCN encoder + GRU pointer decoder** (no PyTorch-Geometric needed): a 2-layer GCN over a per-fold POI graph (geographic kNN $\cup$ training co-visit edges) produces POI features; a GRU seeded by the query rolls out, scoring each POI by inner product (a pointer), masked to stay loop-free with the end reserved for the last hop. A fresh model is trained on every leave-one-out fold.

Classical baselines (counting, CPU)



Learned model: GCN encoder + GRU pointer decoder (GPU)

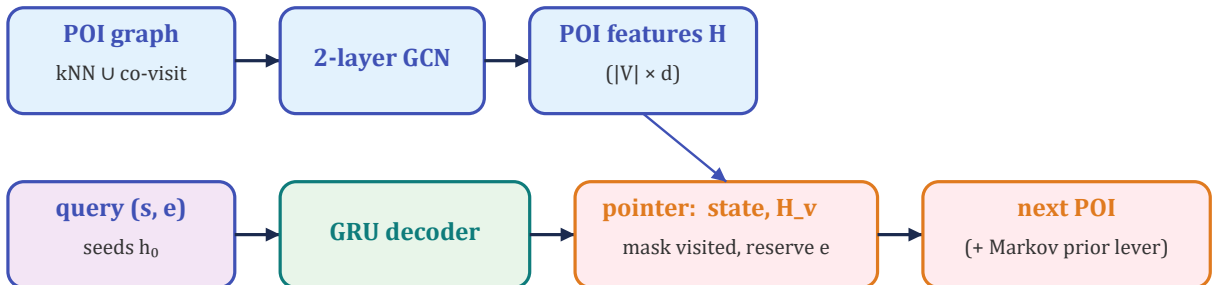


Figure 7 — Top: the classical counting baselines. Bottom: the GCN+pointer architecture. The optional Markov-prior lever blends the classical transition signal into the pointer's logits at decode time.

8. Did we do it right? Reproducing Chen 2016

The decisive check: a **Random** baseline has no modelling choices, so it can only match Chen 2016's Random if our data, protocol and metric are all identical to the paper's. They match within noise on every city — and PoiPopularity matches closely too (near-exactly on Glasgow, Edinburgh, Melbourne). This is the evidence the comparison is valid.

pairs-F1	Toronto	Osaka	Glasgow	Edinburgh	Melbourne
Random — ours	0.298	0.301	0.301	0.270	0.227
Random — Chen 2016	0.310	0.304	0.320	0.261	0.248
PoiPopularity — ours	0.443	0.413	0.510	0.439	0.320
PoiPopularity — Chen 2016	0.384	0.365	0.507	0.436	0.316

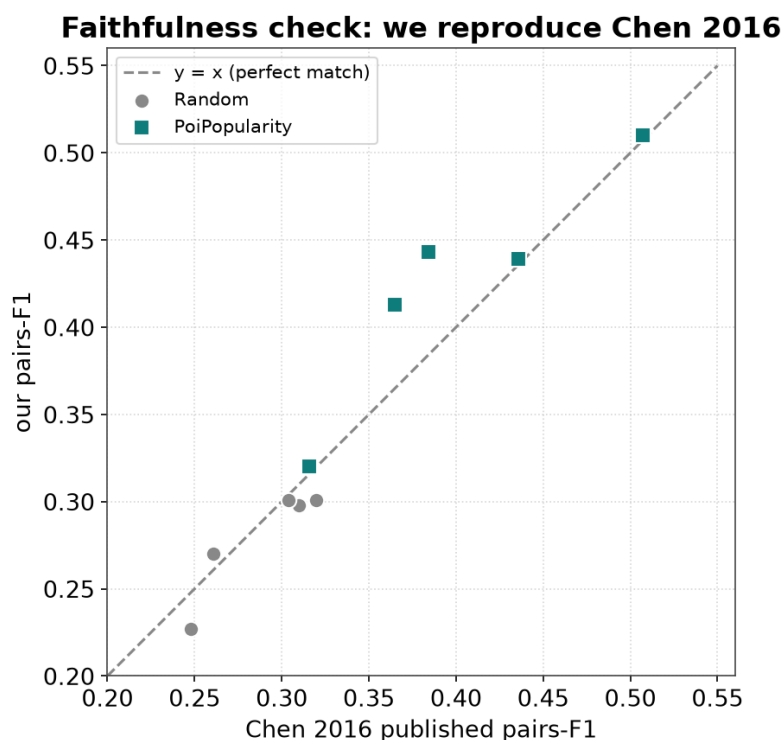


Figure 8 — Our pairs-F1 vs Chen 2016's. Points on the dashed $y = x$ line are exact reproductions; Random sits on the line, confirming a faithful protocol.

9. Results — ours vs the literature

Our measured pairs-F1 (leave-one-out, length ≥ 3). All values are on the published 0–1 scale, versus 0.26–0.29 on Foursquare NYC:

pairs-F1	Toronto	Osaka	Glasgow	Edinburgh	Melbourne
Random	0.298	0.301	0.301	0.270	0.227
PoiPopularity	0.443	0.413	0.510	0.439	0.320
Markov	0.504	0.421	0.587	0.449	0.333

MarkovPath 0.528 0.398 0.543 0.452 0.346

Table 2 — Our classical pairs-F1 per city (leave-one-out, length ≥ 3).

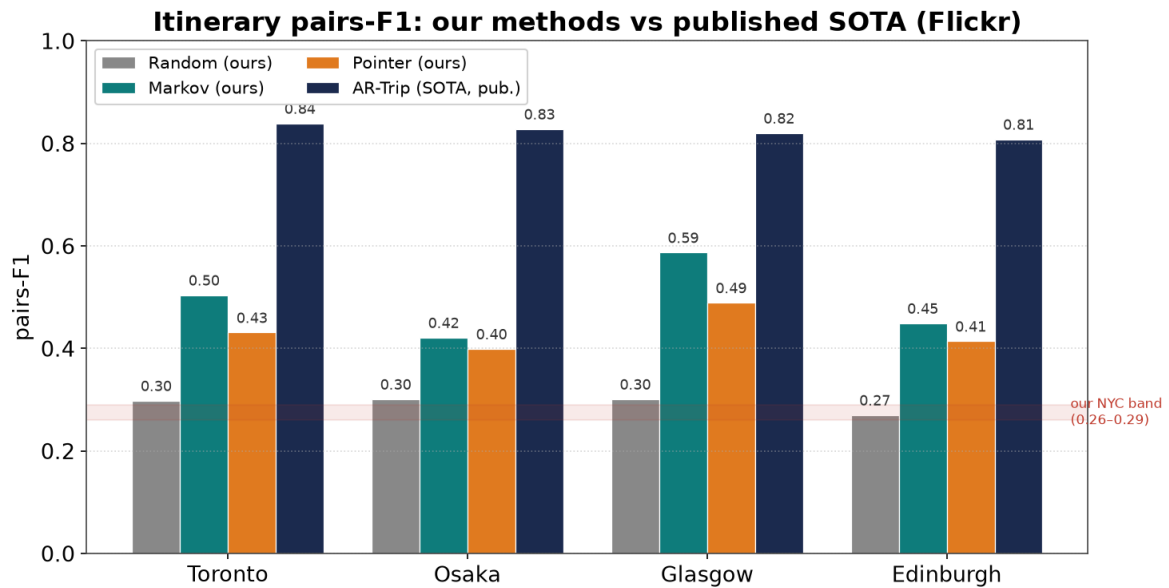


Figure 9 — Our methods beside the published SOTA (AR-Trip). Everything is on the literature's scale; the red band is where our NYC itinerary numbers sat.

The directly-comparable published numbers (same protocol, 0–1 scale):

pairs-F1 (published)	Toronto	Osaka	Glasgow	Edinburgh
PoiRank (Chen)	0.518	0.511	0.548	0.432
Rank+Markov (Chen)	0.512	0.486	0.545	0.444
DeepTrip	0.748	0.755	0.782	0.660
CTLTR	0.748	0.719	0.763	0.681
SelfTrip	0.835	0.851	0.818	0.779
AR-Trip	0.839	0.828	0.820	0.808

Table 3 — Published pairs-F1 (curated comparable subset). No single SOTA across all cities: AR-Trip leads on Toronto/Glasgow/Edinburgh, SelfTrip on Osaka (0.851).

10. The learned model — measured result and the honest finding

	Toronto	Osaka	Glasgow	Edinburgh	Melbourne
Pointer (beam) pairs-F1	0.431	0.399	0.489	0.414	0.312
Pointer point-F1	0.705	0.684	0.734	0.689	0.614
ref: Markov (ours)	0.504	0.421	0.587	0.449	0.333

Honest reading. The light from-scratch pointer reaches pairs-F1 **0.31–0.49** — **on the published scale and clearly above Random** — and its point-F1 (~0.68–0.73) shows it recovers the *right POIs*. But its

ordering trails the simple Markov baseline on every city, and it is far below the neural SOTA band (0.66–0.85). This is the same lesson as Strategy B on NYC: *naively training a dedicated pointer on the tiny per-fold data does not beat a strong simple model*. The published neural methods reach SOTA with machinery our light model omits — self-supervised / contrastive pre-training on the unlabelled trajectories, trajectory augmentation, and adversarial training.

Levers to close the gap (implemented, opt-in in PointerConfig, reported side-by-side in the notebook): a decode-time **Markov transition prior** (blends the fold's empirical log $P(j|i)$ into the pointer logits — directly targets the weak ordering; a quick Osaka check lifts pairs-F1 from 0.42 to 0.44, edging past Markov), an optional **user embedding**, and **early stopping** on an internal validation split.

11. Honest deviations and limitations

- **Markov $\neq$ Chen's Markov.** Ours is a raw empirical first-order transition matrix (Laplace-smoothed); Chen's is feature-factored. Ours therefore differs and on the data-rich cities exceeds Chen's published Markov — a modelling difference, not a protocol bug. Random and PoiPopularity are the clean reproductions.
- **PoiRank / Rank+Markov are cited, not re-implemented** (they need a per-query rankSVM); their numbers come straight from Chen 2016.
- **Excluded as not comparable:** POIBERT (80/20 split, percentage F1, no pairs-F1), DLIR 2025 (8-hour split, different F1 ≈ 0.49), TourEmbedding (no pairs-F1, percentage-scale F1, only the first POI given, no leave-one-out).
- **Reproducibility:** classical numbers reproduce to ± 0.01 across NumPy versions (RNG/tie-break on these tiny datasets) — well within the papers' ± 0.15 – 0.25 std deviations.

12. How to run

Locally (CPU, classical — real numbers in minutes):

```
py -3.11 -m src.flickr.run_flickr --data_dir data/flickr
```

Colab (GPU, the learned pointer): open colab_flickr.ipynb — it clones the repo, downloads the five cities, runs the classical baselines, then trains the GCN+pointer per leave-one-out fold (pure and Markov-prior+user variants) and prints the combined comparison vs the published numbers. No PyTorch-Geometric needed.

Tests: `py -3.11 -m pytest` — 79 cases, including the pairs-F1 protocol pinned against Chen 2016's reference, the leave-one-out splitter, and baseline / pointer invariants.

Appendix A — Code map

Module	What it does
src/flickr/data.py	load Chen (traj-/poi-) & Lim (userVisits-/POI-) formats → FlickrCity
src/flickr/evaluate.py	leave-one-out CV harness; F1 + pairs-F1; loop-free filter
src/flickr/baselines.py	Random / PoiPopularity / Markov / MarkovPath + factories
src/flickr/pointer.py	self-contained GCN+pointer; per-fold trainer; Markov-prior/user levers

src/flickr/published.py	curated comparable literature numbers + exclusion notes
src/flickr/run_flickr.py	orchestration + ours-vs-published tables + CLI
tests/test_flickr.py	loader, trajectory build, pairs-F1 vs Chen, splitter, baselines

Appendix B — Key references

- Chen, Ong, Xie. *Learning Points and Routes to Recommend Trajectories*. CIKM 2016 — origin of pairs-F1 + our data.
- Gao et al. *DeepTrip*. SIGSPATIAL 2019. · Gao et al. *SelfTrip*. KBS 2022.
- Rashid et al. *DeepAltTrip*. TKDE 2021. · Shu et al. *AR-Trip*. SIGIR 2024.
- Lim et al. *Tour Recommendation and Trip Planning using LBSNs: A Survey*. KAIS 2019.
- Krichene, Rendle. *On Sampled Metrics for Item Recommendation*. KDD 2020 — why cross-protocol metrics are inconsistent.